Understanding Pandas

What Pandas Really Is, and How We Use It in Feature Engineering

Class Notes · BCA-3004T Feature Engineering · Unit I

1. What is Pandas, Really?

Pandas is a Python library built for working with labelled, tabular data — the kind of data you already know from Excel: rows and columns, with headers that mean something.

> *NumPy gives you a grid of numbers. Pandas gives that grid a memory — it remembers what each row and column is called.*

A NumPy array only knows positions: row 0, column 3. It has no idea that column 3 is 'Fare' or that row 0 is passenger 'Braund, Mr. Owen Harris'. Pandas sits on top of NumPy and adds exactly that missing layer — labels. This is why Pandas, not raw NumPy, is the tool we reach for the moment our data has meaning attached to it: names, dates, categories, IDs.

For Feature Engineering specifically, Pandas is the workbench. Every technique we have studied so far — binning, encoding, scaling, handling missing values, feature selection — is applied to data sitting inside a Pandas DataFrame. Before we can engineer a feature, we must first be able to load it, look at it, select it, and reshape it. That is what this chapter is about.

2. The Two Core Structures: Series and DataFrame

Pandas is built on just two data structures. Understanding these two, deeply, is 80% of learning Pandas.

2.1 Series — a single labelled column

A Series is a one-dimensional array with a label attached to every value. Think of it as one column pulled out of an Excel sheet, where the row numbers on the left are the index.

```
import pandas as pd

ages = pd.Series([22, 38, 26, 35], name='Age')
print(ages)
```

2.2 DataFrame — the full table

A DataFrame is a two-dimensional table: many Series (columns) stitched together, all sharing the same index. This is the structure you will use for almost every real task.

```
df = pd.read_csv('titanic.csv')
print(type(df))          # <class 'pandas.core.frame.DataFrame'>
print(type(df['Age']))   # <class 'pandas.core.series.Series'>
```

> *A DataFrame is just a dictionary of Series that all agree to share the same row labels.*

Aspect	Series	DataFrame
Dimensions	1-D (a single column of values)	2-D (rows and columns, like a table)
Index	One label per value	One label per row, one name per column
Analogy	A single labelled column in Excel	An entire Excel sheet
Built from	A list, array, or dict	A dict of Series, a 2-D array, or a CSV file

3. Loading Data and Taking a First Look

We will use the Titanic dataset throughout, exactly as in our earlier NumPy sessions, so the numbers stay familiar and the focus stays on the Pandas operations.

```
df = pd.read_csv('titanic.csv')

df.head()        # first 5 rows — always look before you touch
df.tail(3)       # last 3 rows
df.shape         # (891, 12) -> 891 rows, 12 columns
df.columns       # names of every column
df.info()        # column names, data types, non-null counts
df.describe()    # count, mean, std, min, quartiles, max — numeric columns only
```

> *df.info() is the single most important line in this entire chapter — it tells you data types and missing values in one shot, before you write a single feature engineering line.*

4. Selecting Rows and Columns

4.1 Selecting columns

```
df['Age']                # a single column -> returns a Series
df[['Age', 'Fare']]      # multiple columns -> returns a DataFrame
```

4.2 Selecting rows: loc and iloc

This is the part students confuse the most, so remember it as a simple rule:

> *loc uses the names on the labels. iloc uses the position, counted from zero — exactly like a NumPy array.*

```
df.loc[0]                # row labelled 0
df.loc[0:3]              # rows labelled 0,1,2,3  -> end IS included
df.iloc[0:3]             # rows at position 0,1,2 -> end is EXCLUDED
df.loc[0, 'Fare']        # single cell, by label
```

```
df.iloc[0, 9]            # single cell, by position
```

	.loc[ ]	.iloc[ ]
Works with	Labels (index names, column names)	Integer positions (0, 1, 2, ...)
Example	df.loc[3, 'Age']	df.iloc[3, 5]
Slicing	End label is included	End position is excluded (Python style)

5. Filtering Rows: Boolean Indexing

Filtering is how we ask questions of the data. It is also the operation that most feature engineering rules are built on — for example, deciding which rows belong to a bin, or which rows count as outliers.

```
df[df['Age'] > 60]                        # passengers older than 60
df[(df['Sex'] == 'female') & (df['Pclass'] == 1)]   # combine conditions with &
and |
df['Age'].mean()                           # works the same as on a NumPy array
```

> *df['Age'] > 60 does not return the data — it returns a Series of True/False. Passing that Series back into df[...] is what actually filters the rows.*

6. Handling Missing Data

Real datasets are never complete. The Titanic dataset itself has missing Age, Cabin, and Embarked values — this is exactly the MCAR/MAR/MNAR problem we studied earlier, now seen inside Pandas.

```
df.isnull().sum()              # count of missing values per column
df.dropna(subset=['Embarked'])  # drop rows missing Embarked
df['Age'].fillna(df['Age'].median(), inplace=True)   # impute with median
```

Method	What it Does	When to Use
isnull() / isna()	Flags missing cells as True/False	First step — always audit before deciding
dropna()	Removes rows/columns with missing values	Missingness is small and random (MCAR)
fillna(value)	Replaces missing values with a constant, mean, median or mode	You want to preserve every row
fillna(method='ffill'/'bfill')	Carries forward/backward the nearest known value	Ordered/time-series style data

7. GroupBy — Split, Apply, Combine

GroupBy is Pandas' most powerful feature engineering tool. It follows one pattern every time: split the data into groups, apply a calculation to each group, then combine the results back together.

```
df.groupby('Pclass')['Fare'].mean()
# Pclass
# 1    84.15
# 2    20.66
# 3    13.68

df.groupby('Sex')['Survived'].mean()   # survival rate by gender
```

> *Whenever a feature engineering question contains the word 'by' or 'per' — average fare per class, survival rate by gender — the answer is a groupby.*

8. Everyday Exploration Tools

```
df['Embarked'].value_counts()      # frequency of each category – vital before
encoding
df.sort_values('Fare', ascending=False).head()   # top 5 highest fares
df['Age'].apply(lambda x: 'Adult' if x >= 18 else 'Minor')   # row-wise
transformation
```

value_counts() deserves special attention: before you one-hot encode or label-encode any categorical column, always run value_counts() first, to see how many categories exist and whether any category is rare enough to need grouping.

9. Why All of This Matters for Feature Engineering

- Binning and Discretization — built using boolean indexing and pd.cut(), which relies on the selection skills above.
- Encoding (One-Hot, Label) — always preceded by value_counts() to check category frequency and cardinality.
- Missing Value Treatment — entirely a Pandas operation: isnull(), dropna(), fillna().
- Feature Selection and Aggregation — groupby is how we compute per-category statistics used as new features.
- Any technique from sklearn — StandardScaler, encoders, imputers — takes a Pandas DataFrame or column as its input in practice.

> *You cannot skip Pandas and go straight to sklearn. Every feature engineering pipeline begins with a DataFrame.*

10. Key Takeaways

- Pandas adds labels on top of NumPy's grid of numbers — that is its entire purpose.
- Series = one labelled column. DataFrame = many Series sharing one index.
- loc works on labels; iloc works on integer positions, with the end excluded.

- Boolean indexing — df[condition] — is how we filter rows, and underlies most feature engineering rules.
- Always run df.info(), df.isnull().sum(), and value_counts() before applying any feature engineering technique.
- groupby follows split → apply → combine, and answers any 'by/per category' question.

Next session: concat() for combining datasets, and pivot tables for reshaping grouped summaries into a table format.